

Aula 01

Prof. Dr. Alencar Machado

Agenda

- Spring Boot
 - Configurações do projeto
- Migrações com Flyway
 - Criação da tabela Aluno
- Spring Data JPA
 - Mapeamento da entidade Aluno

Spring Boot (start.spring.io)

 **spring** initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.4.0 (SNAPSHOT)

☐ 3.4.0 (M3)

☐ 3.3.5 (SNAPSHOT)

☐ 3.3.4

☐ 3.2.11 (SNAPSHOT)

☒ 3.2.10

Project Metadata

Group

br.csi

Artifact

avaliador-de-projetos

Name

Avaliador de Projetos

Description

api para avaliar projetos

Package name

br.csi.avaliador-de-projetos

Packaging

☒ Jar

☐ War

Java

☐ 23

☐ 21

☒ 17

Dependencies

ADD DEPENDENCIES... ⌘ + B

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Lombok

DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

PostgreSQL Driver

SQL

A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

Spring Data JPA

SQL

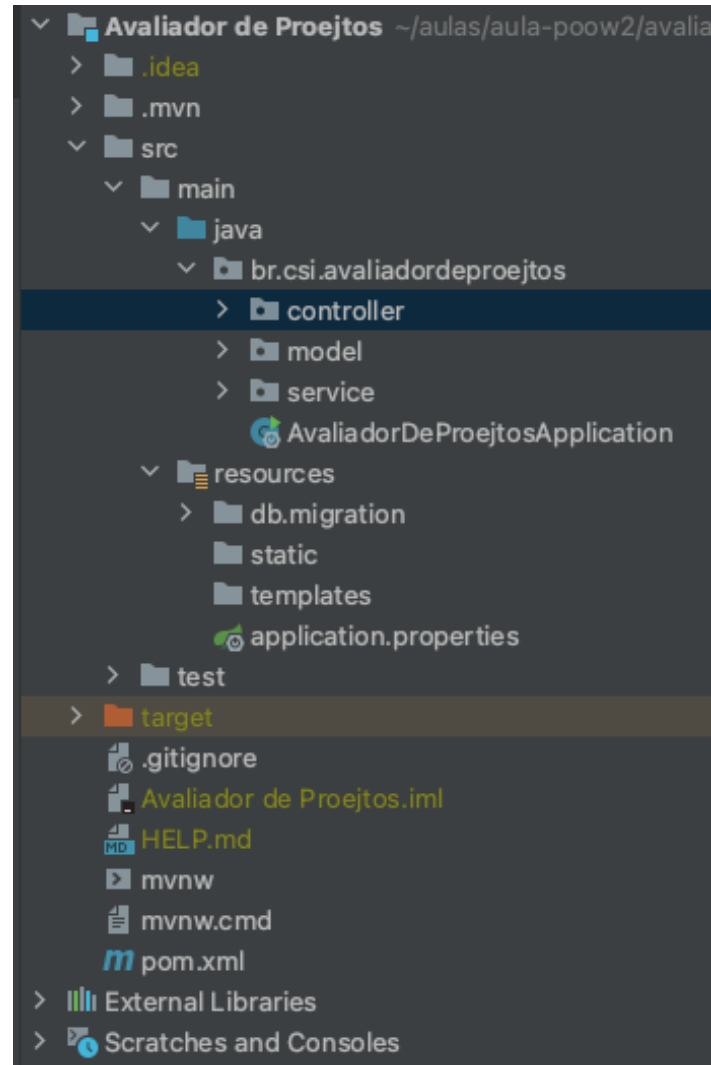
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Flyway Migration

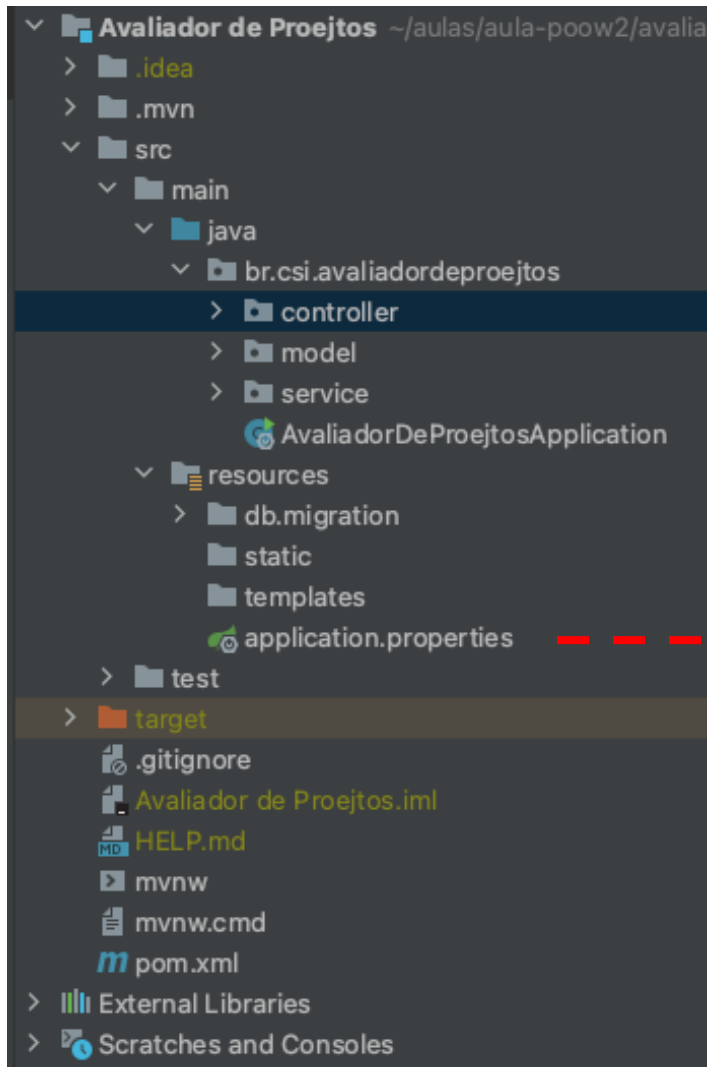
SQL

Version control for your database so you can migrate from any version (incl. an empty database) to the latest version of the schema.

Spring Boot (ESTRUTURA PROJETO)



Spring Boot (**application.properties**)



```
server.servlet.context-path=/avaliador-de-projetos

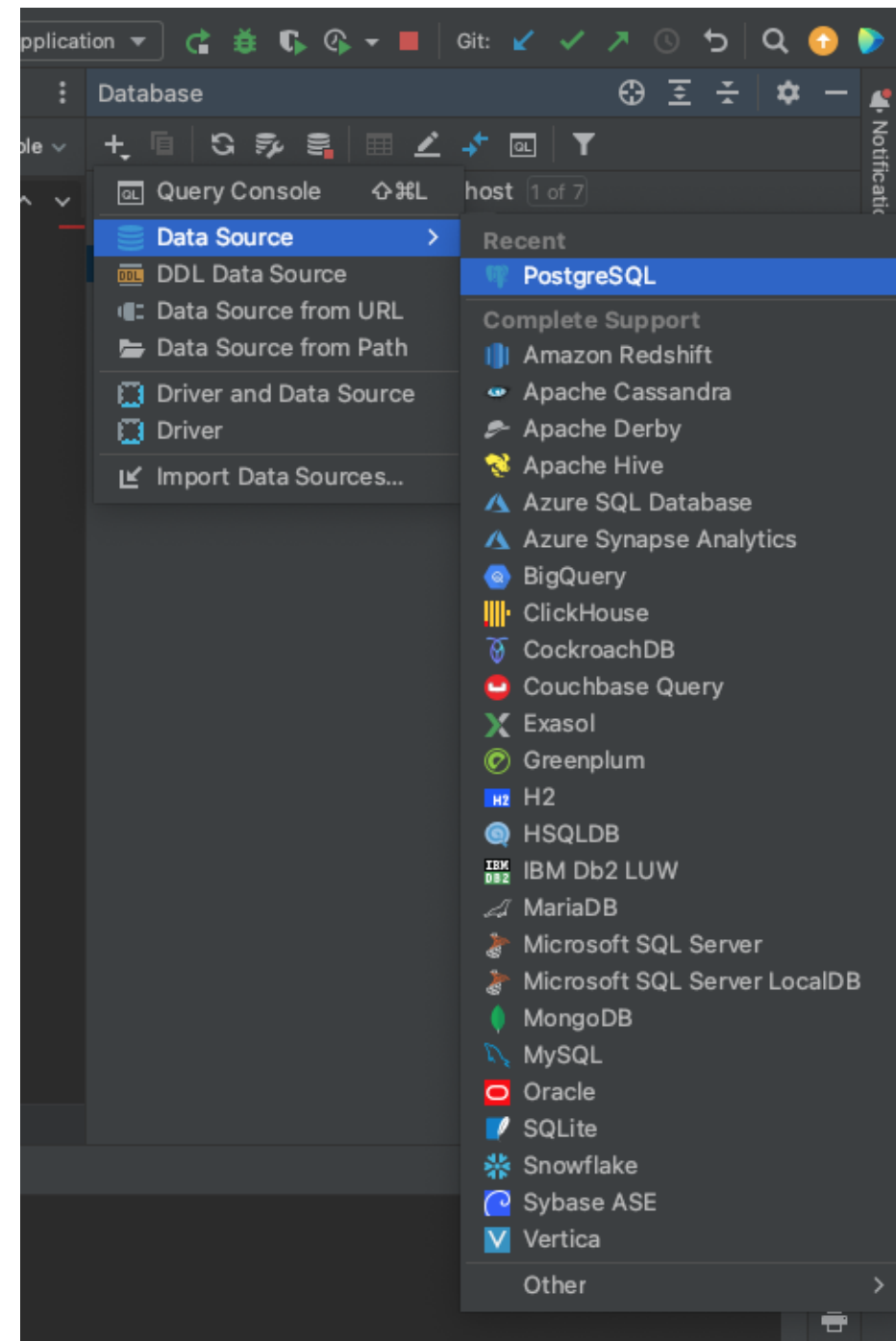
spring.datasource.url=jdbc:postgresql://localhost:5432/avaliadordeprojetos
spring.datasource.username=alencarmachado
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Criar banco “**avaliadordeprojetos**”

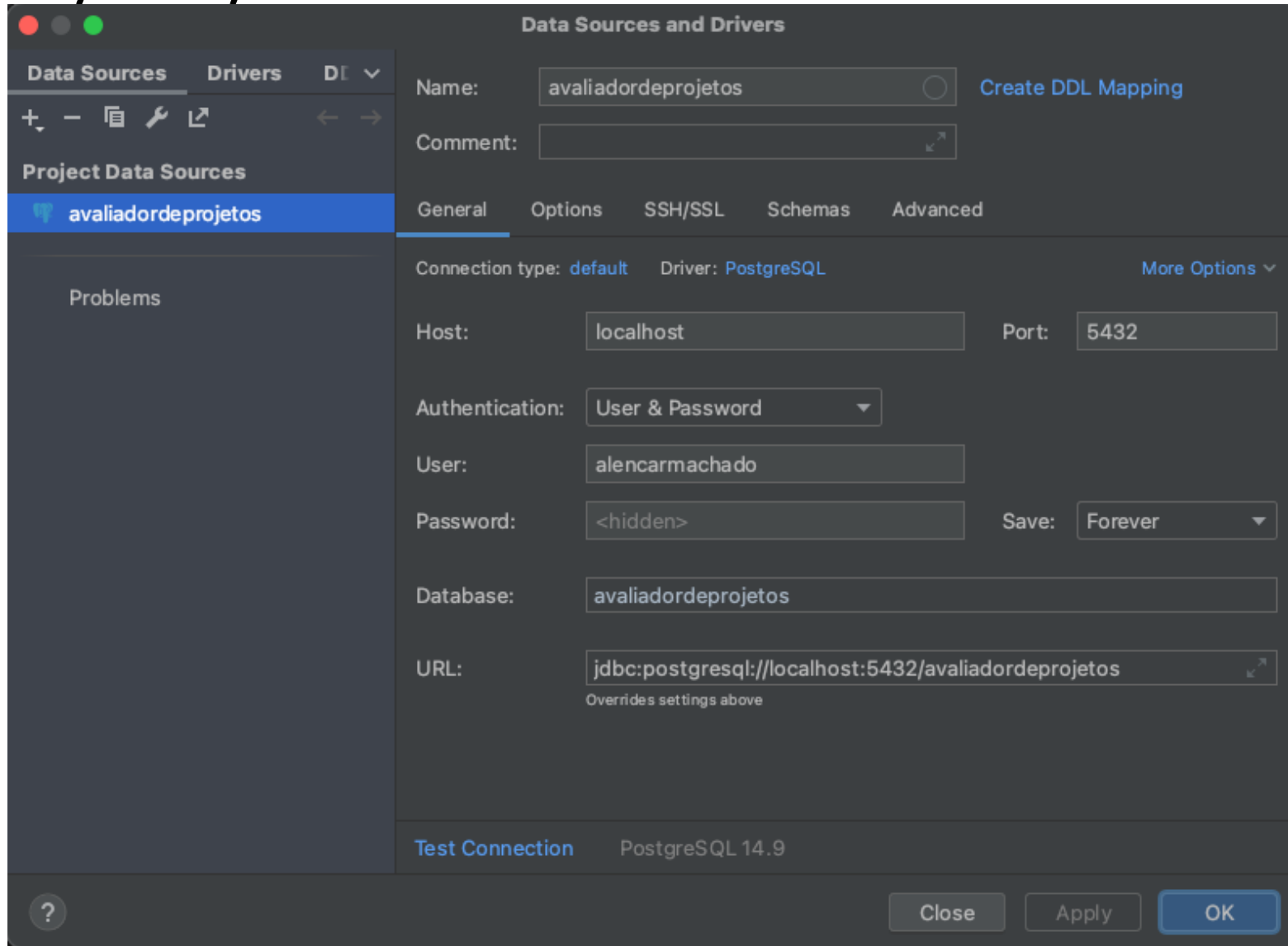
Migrations com Flyway

- Aba Database
 - Adicionar Data Source
 - Configurar o banco

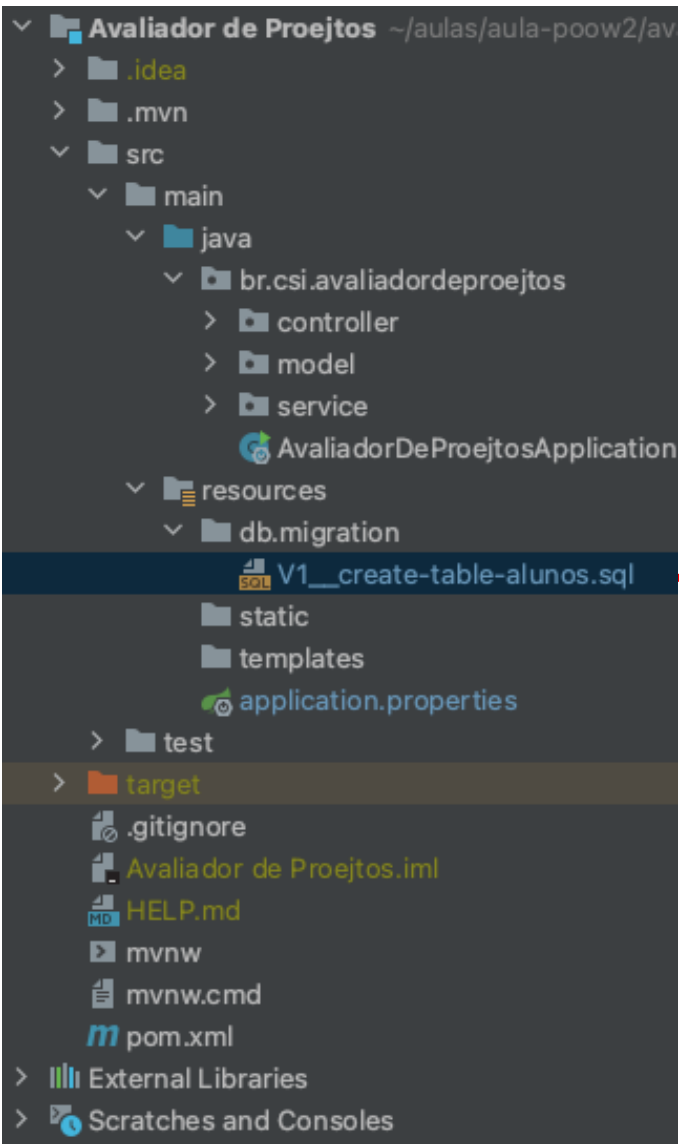


Migrations com Flyway

- Aba Database
 - Adicionar Data Source
 - Configurar o banco



Flyway (V1__create-table-alunos.sql)



```
create table alunos(  
    id serial not null primary key,  
    uuid UUID DEFAULT gen_random_uuid(),  
    nome varchar(50) not null,  
    matricula varchar(8) not null unique,  
    email varchar(100) not null unique,  
    bairro varchar(100) not null,  
    cep varchar(9) not null,  
    complemento varchar(100),  
    numero varchar(20),  
    uf char(2) not null,  
    cidade varchar(100) not null  
)
```

Migrations com Flyway

The screenshot shows a database IDE interface. The top toolbar contains a red '1º' next to the 'Query Console' icon. The left sidebar shows a tree view of the database structure, with a red '2º' next to the 'flyway_schema_history' table. The main editor area contains a SQL query: `select * from flyway_schema_history`, with a red '3º' next to the 'select' keyword. The bottom panel shows the output of the query, which is a table with 6 columns: `installed_rank`, `version`, `description`, `type`, `script`, and `checksum`. The first row of data is: `1 1 create-table-alunos SQL V1__create-table-alunos.sql -`. A green arrow points to the first row of the output table.

Query Console

New Query Console

Open Default Console

Open

console

All

Output

avaliadordeprojetos....flyway_schema_history

installed_rank	version	description	type	script	checksum
1	1	create-table-alunos	SQL	V1__create-table-alunos.sql	-

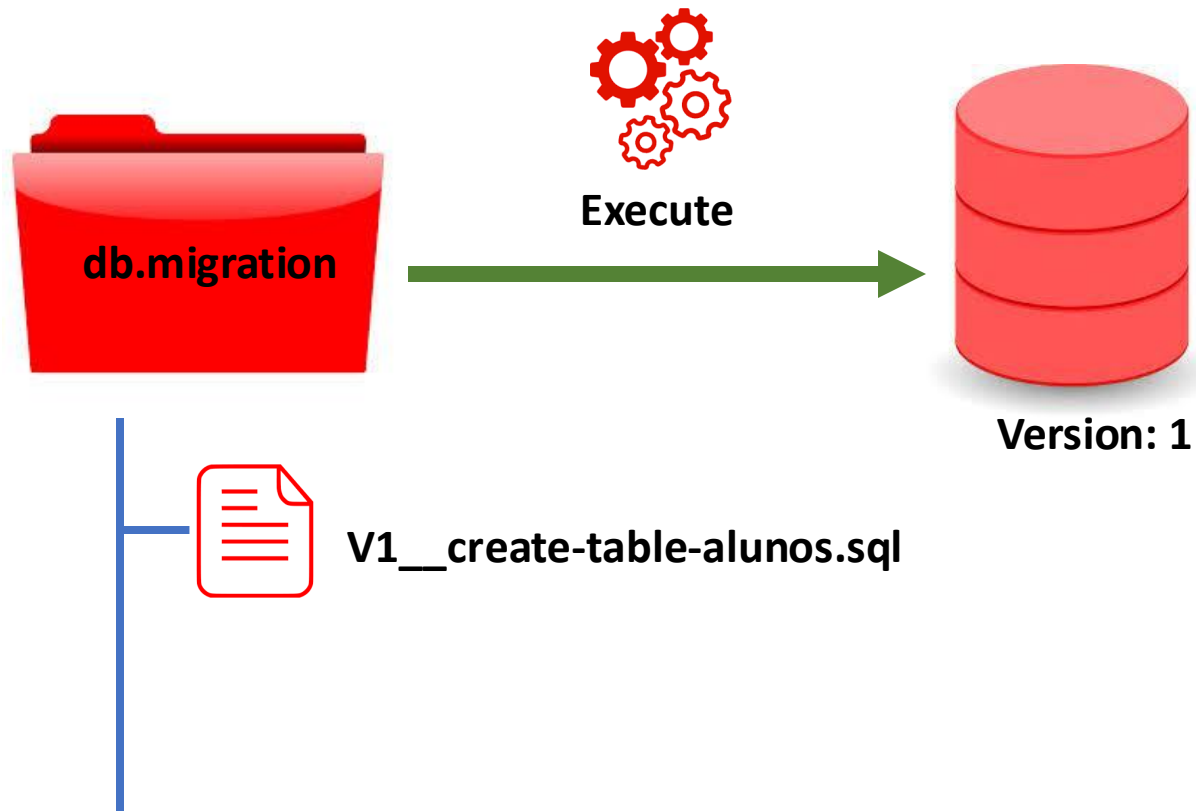
Migrations com Flyway

- <https://flywaydb.org/>
- `select * from flyway_schema_history`

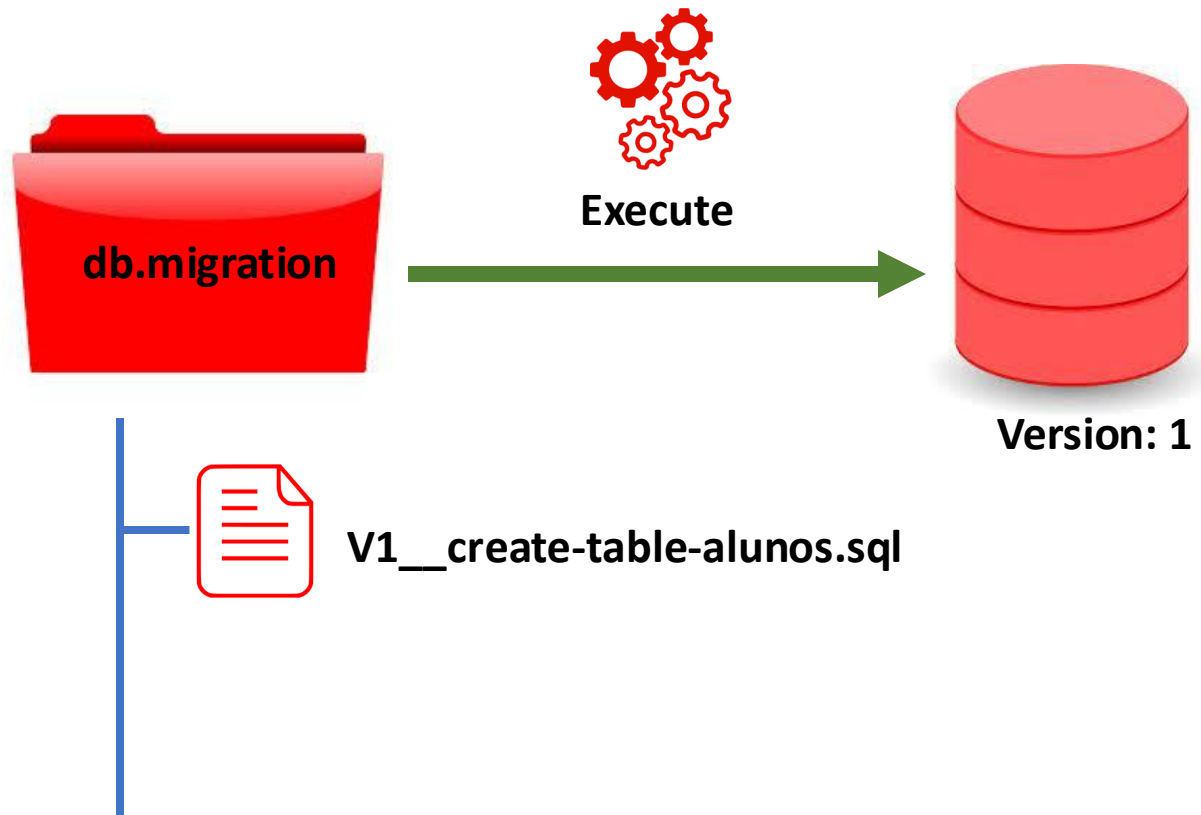
version	description	type	script	checksum	Installed_by	Installed_on	Success
1	create-table-alunos	SQL	V1__create-table-alunos.sql	-2062151907	alencarmachado	2023-08-20 07:14:54.001583	true

Migrations com Flyway

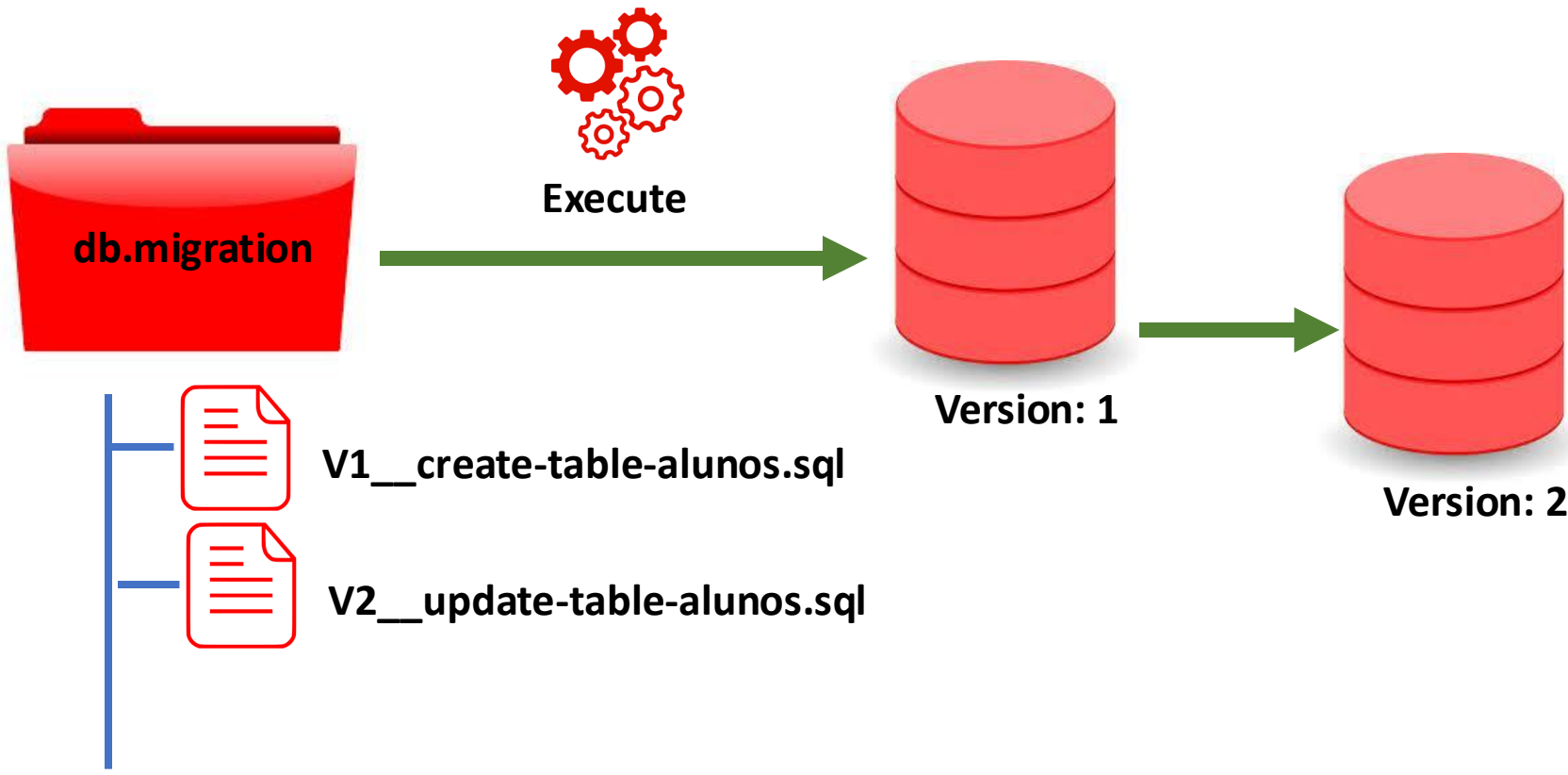
version	description	type	script	checksum	Installed_by	Installed_on	Success
1	create-table-alunos	SQL	V1__create-table-alunos.sql	-2062151907	alencarmachado	2023-08-20 07:14:54.001583	true



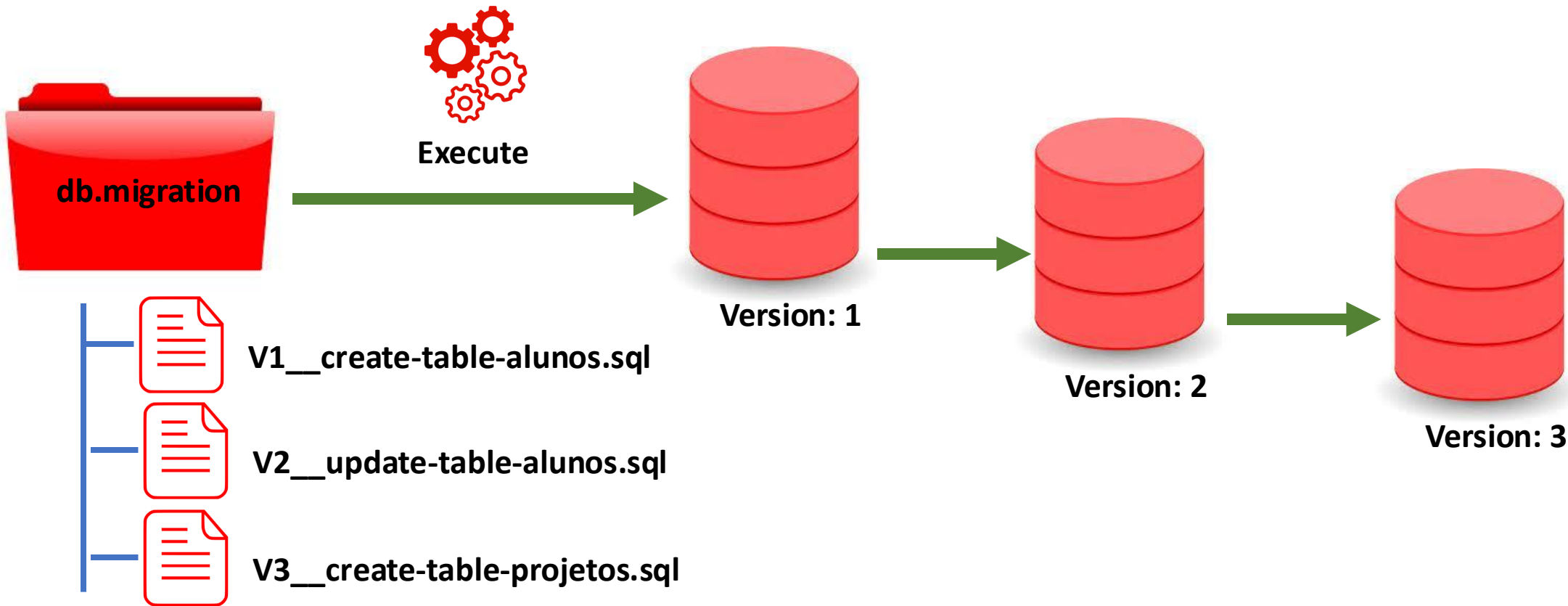
version	description	type	script	checksum	Installed_by	Installed_on	Success
1	create-table-alunos	SQL	V1__create-table-alunos.sql	-2062151907	alencarmachado	2023-08-20 07:14:54.001583	true



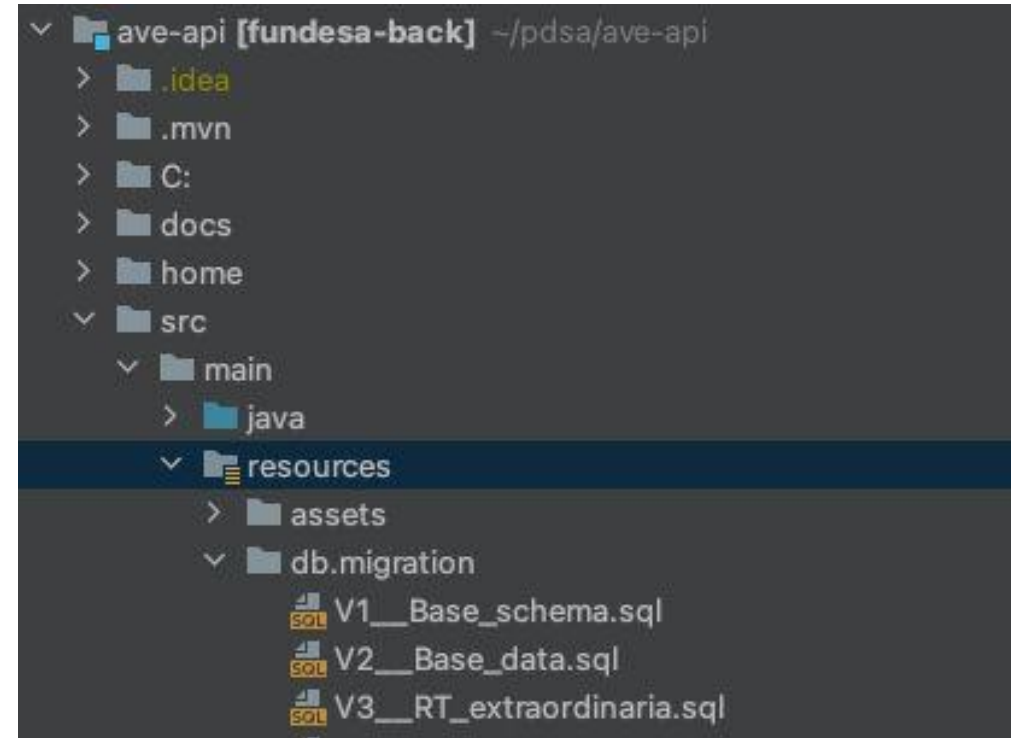
version	description	type	script	checksum	Installed_by	Installed_on	Success
1	create-table-alunos	SQL	V1__create-table-alunos.sql	-2062151907	alencarmachado	2023-08-18 18:10:33.002299	True
2	update-table-alunos	SQL	V2__update-table-aluno .sql	-695580231	alencarmachado	2023-08-19 07:14:54.0180810	True



version	description	type	script	checksum	Installed_by	Installed_on	Success
1	create-table-alunos	SQL	V1__create-table-alunos.sql	-2062151907	alencarmachado	2023-08-18 18:10:33.002299	True
2	update-table-alunos	SQL	V2__update-table-aluno .sql	-695580231	alencarmachado	2023-08-19 07:14:54.0180810	True
3	create-table-projetos	SQL	V2__create-table-projetos .sql	-687381716	alencarmachado	2023-08-20 07:14:54.001583	True

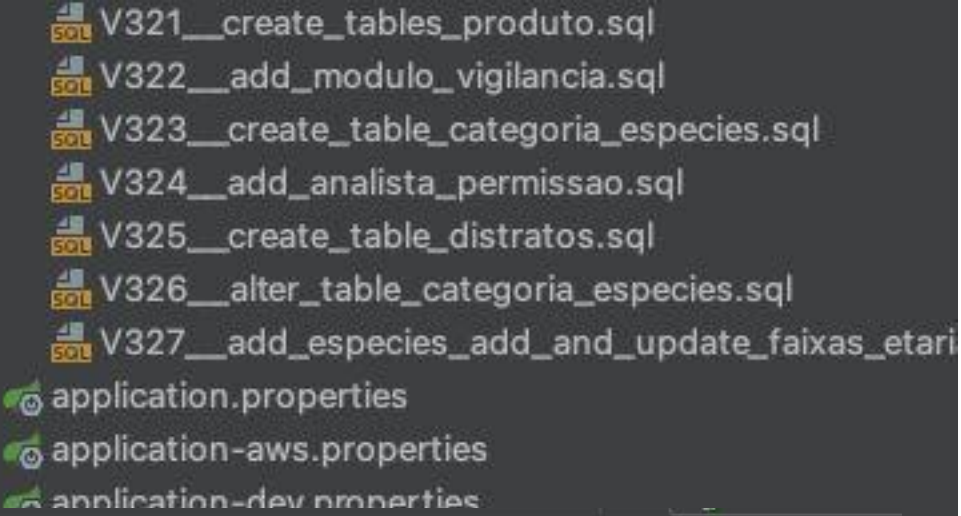


PDSA: API Aves (2020 +/-)



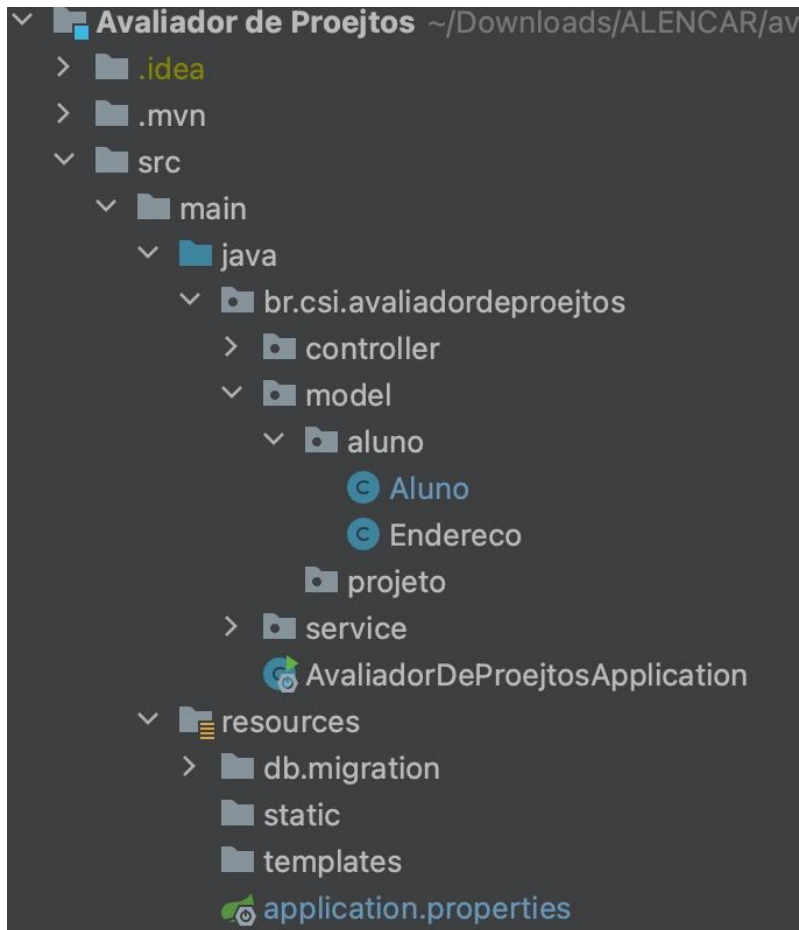
schema_version [Helpdesk]									
WHERE									
ORDER BY installed_rank									
		type	script	checksum	installed_by	installed_on			
1		SQL	V1__Base_schema.sql	-199120115	postgres	2020-03-09 00:19:36.383984			
2		SQL	V2__Base_data.sql	-972605329	postgres	2020-03-09 00:19:36.924967			
3		SQL	V3__RT_extraordinaria.sql	1949162815	postgres	2020-03-09 00:19:36.972843			
4		SQL	V4__Log_upload_laudo.sql	1584133515	postgres	2020-03-09 00:19:36.985751			
5		SQL	V5__Sve_modulos.sql	-1591017013	postgres	2020-03-09 00:19:36.999013			
6		SQL	V6__modificacao_status.sql	-1224907139	postgres	2020-03-09 00:19:37.009412			
7	cacao	SQL	V7__RT_MAPA__Solicitar_certificacao.sql	-896592446	postgres	2020-03-09 00:19:37.019392			
8		SQL	V8__Unificacao_granjas.sql	-1077784318	postgres	2020-03-09 00:19:37.040600			

PDSA: API Aves (2020 +/-)



schema_version [Helpdesk] x							
Tx: Auto v DDL Q							
WHERE ORDER BY installed_rank							
		type	script	checksum	installed_by	installed_on	
268		SQL	V316__add_modulos_uni_inpe.sql	-846551942	pdsaadmin	2025-01-17 12:12:55.663242	
269		SQL	V317__add_zootecnista_permissao.sql	-618800137	pdsaadmin	2025-01-23 16:14:25.466600	
270	rdinaria	SQL	V318__log_motivo_exclusao_extraordinaria.sql	1115373221	pdsaadmin	2025-02-17 15:45:59.509758	
271		SQL	V319__alter_table_usuario.sql	1767225585	pdsaadmin	2025-04-22 09:28:32.900757	
272		SQL	V320__alter_table_cidade.sql	1489425332	pdsaadmin	2025-04-22 09:28:32.983144	
273		SQL	V321__create_tables_produto.sql	1547090224	pdsaadmin	2025-05-15 10:13:56.920222	
274		SQL	V322__add_modulo_vigilancia.sql	1193597377	pdsaadmin	2025-06-06 15:22:40.226351	
275	ecies	SQL	V323__create_table_categoria_especies.sql	1612023531	pdsaadmin	2025-07-01 13:13:42.966694	
276		SQL	V324__add_analista_permissao.sql	-2093756584	pdsaadmin	2025-07-29 17:17:07.636625	
277		SQL	V325__create_table_distratos.sql	-581895241	pdsaadmin	2025-07-29 17:17:07.740334	
278	cies	SQL	V326__alter_table_categoria_especies.sql	-1213242798	pdsaadmin	2025-07-29 17:17:08.270842	
279	e faixas eta...	SQL	V327__add_especies_add_and_update_faixas_etaria...	-1566250852	pdsaadmin	2025-07-29 17:17:08.499522	

Aplicação

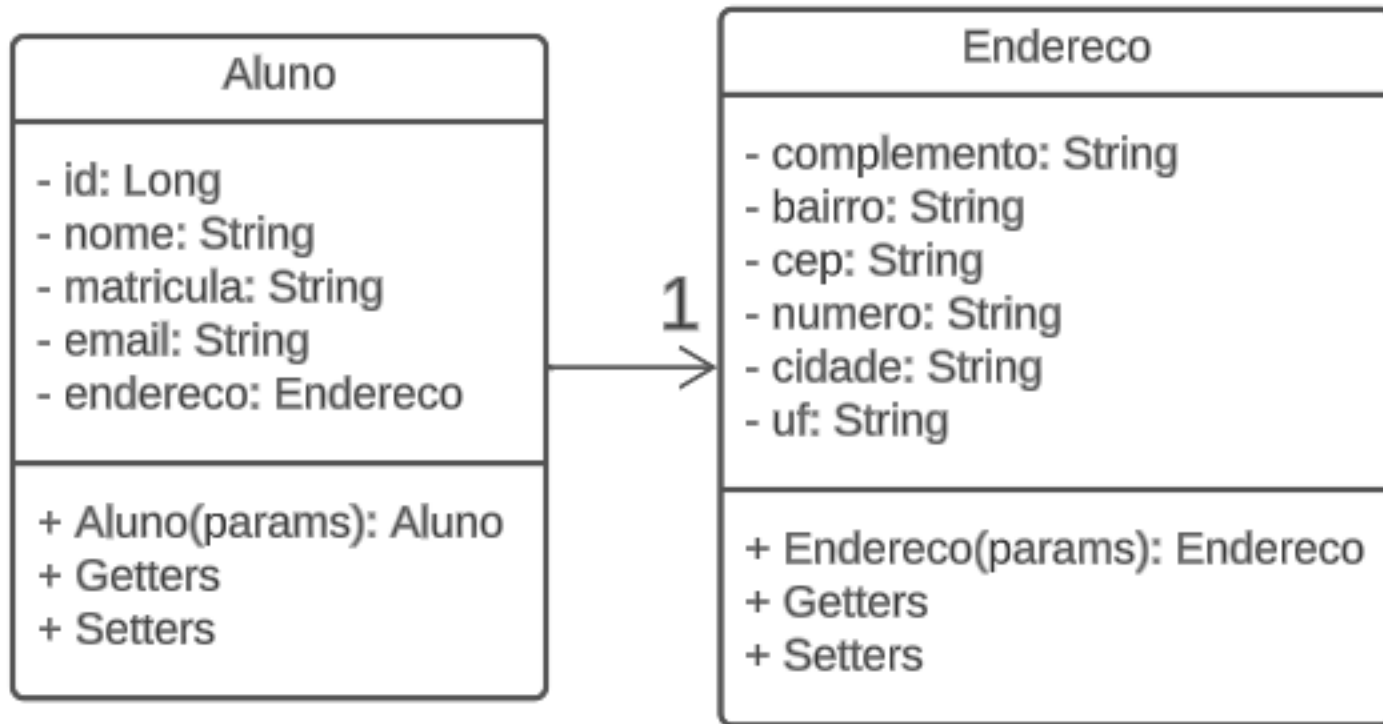


```
public class Aluno {  
    private Long id;  
    private UUID uuid;  
    private String nome;  
    private String matricula;  
    private String email;  
    private Endereco endereco;  
  
    //Setters and Getters  
    //Construtors  
  
}
```

```
public class Endereco {  
  
    private String complemento;  
    private String bairro;  
    private String cep;  
    private String numero;  
    private String cidade;  
    private String uf;  
  
    //Setters and Setters  
    //Construtors  
  
}
```

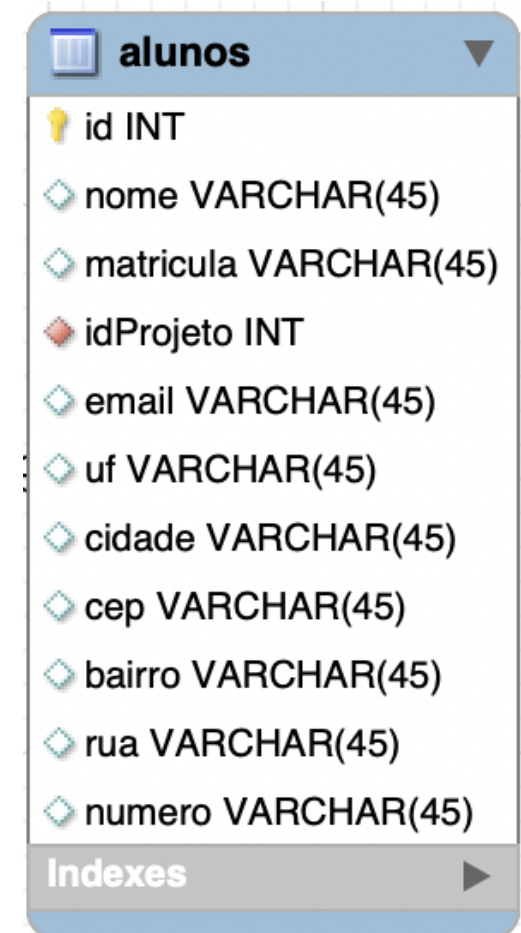
Object Relational Mapping (ORM)

Paradigma Orienta a Objetos

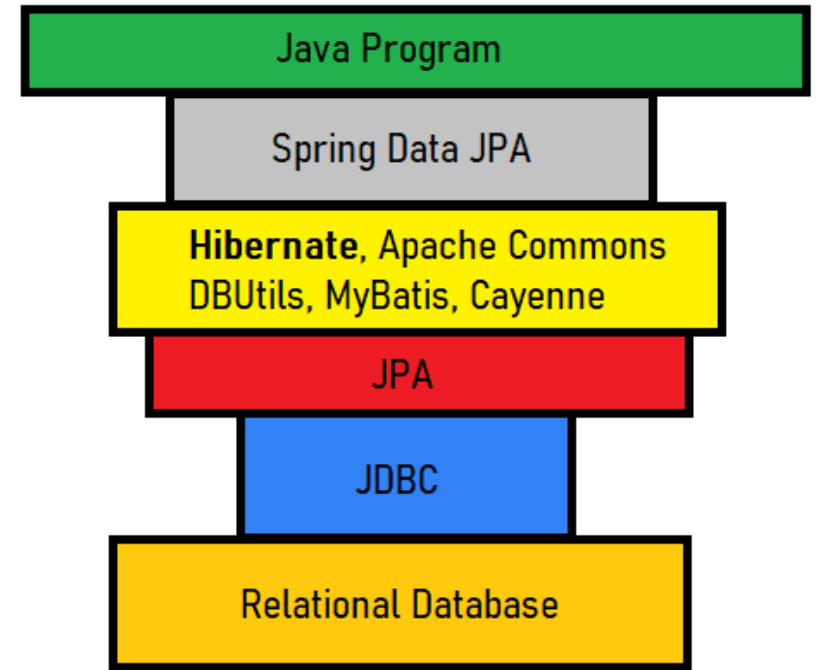
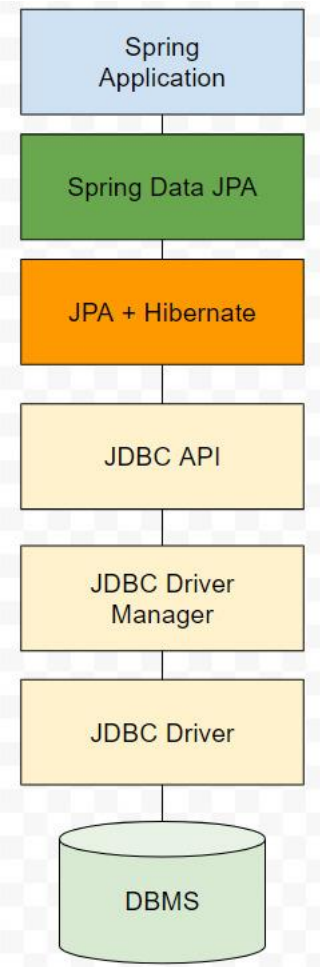
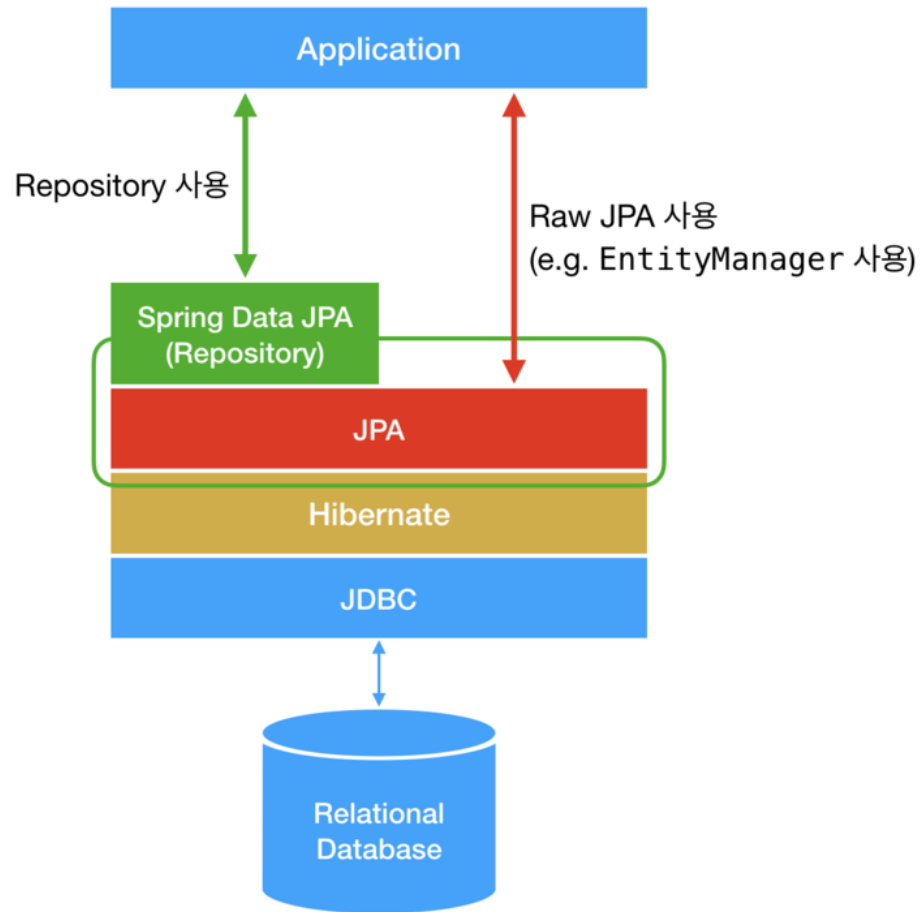


ORM

Paradigma Relacional



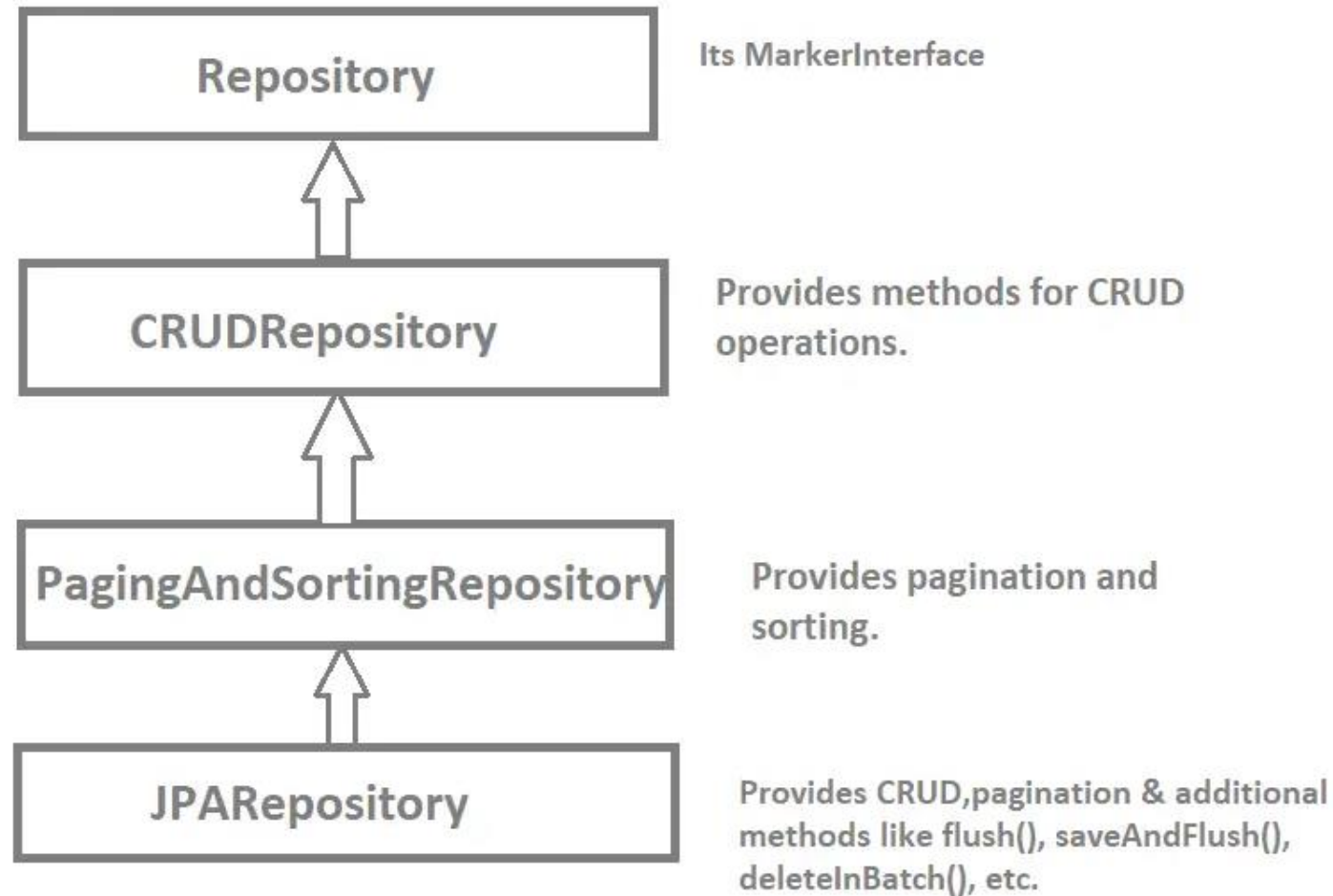
Java Persistence API



Java Persistence API

- Java
 - Hibernate
 - Apache OpenJPA
 - EclipseLink
 - jOOQ
 - Oracle TopLink
- PHP
 - Laravel
 - CakePHP
 - Qcodo
 - RedBeanPHP
- Python
 - Django
 - web2py
 - SQLAlchemy
- .NET
 - NHibernate
 - Entity Framework
 - Dapper

Spring Data JPA



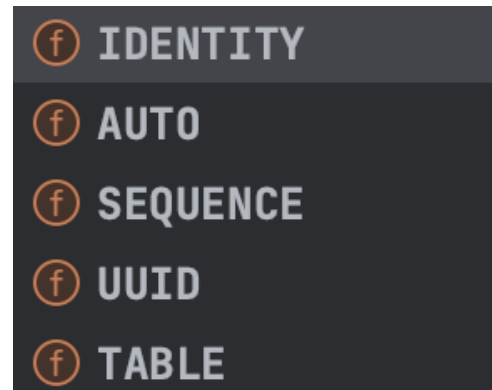
<https://docs.spring.io/spring-data/jpa/reference/>

<https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa>

Spring Data JPA

- **@Entity**
 - Entidades que representam dados que podem ser persistidos em banco de dados. Uma entidade representa uma tabela no BD. Cada instância de uma entidade representa uma linha na tabela.
- **@Table:**
 - Mapear o nome da tabela com a entidade
- **@Id**
 - Anotação que determina a chave primaria da entidade.
- **@GeneratedValue**
 - Utilizado com o @Id para gerar os identificadores através de algumas estratégias:
 - [IDENTITY, SEQUENCE, TABLE, AUTO, UUID]

Spring Data JPA



- @GeneratedValue

- Utilizado com o @Id para gerar os identificadores através de algumas estratégias:
 - **IDENTITY: Espera valores gerados por uma coluna de identidade no banco de dados (auto-increment)**
 - SEQUENCE: Suporta valores gerados através de sequencias (2,4,6,8) sequencia de 2;
 - TABLE: Utilizado com uma tabela separada para manter um contador para PK.
 - AUTO: Permite ao JPA escolher uma estratégia baseada nos recursos do banco de dados
 - UUID: (Universal Unique Identifier) utilizado para gerar um PK de 128 bits
 - e58ed763-928c-4155-bee9-fdbaaadc15f3
 - **Utilizaremos como uma estratégia complementar ao id auto-increment**

<https://www.baeldung.com/hibernate-identifiers>

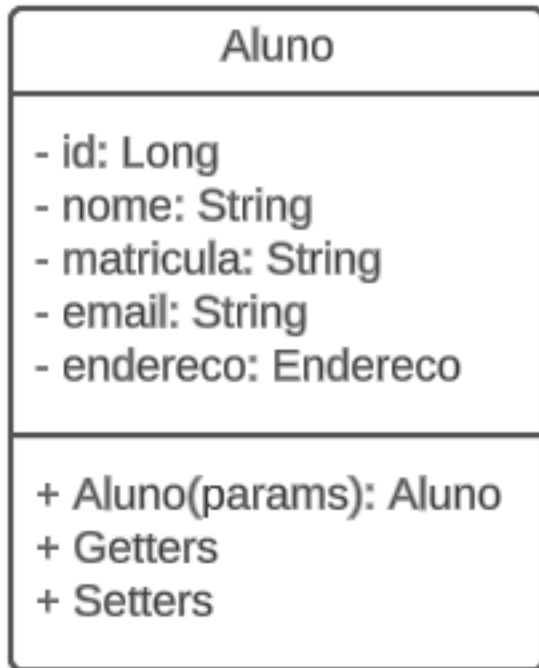
<https://gist.github.com/rponte/bf362945a1af948aa04b587f8ff332f8>

Spring Data JPA

- @Embeddable
 - Utilizado para mapear uma entidade que contém propriedades incorporadas de outra entidade

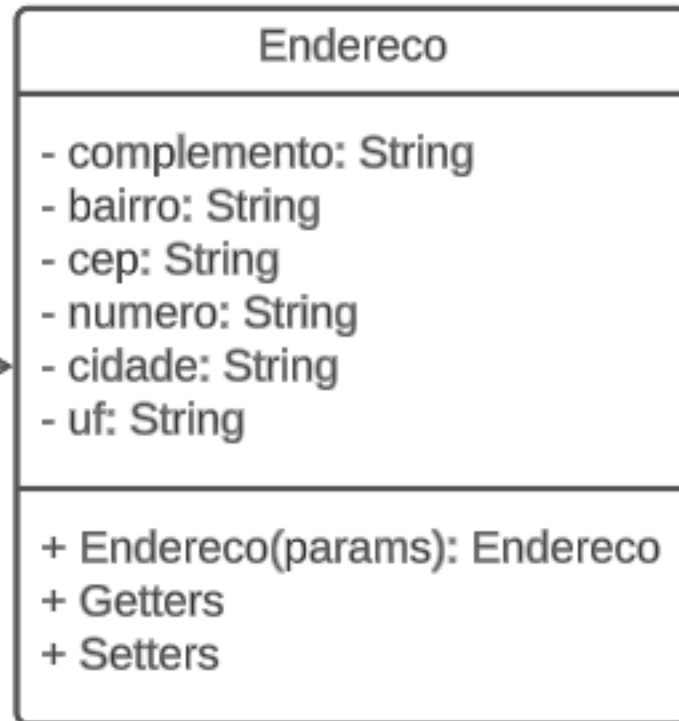
Object Relational Mapping (ORM)

@Embedded



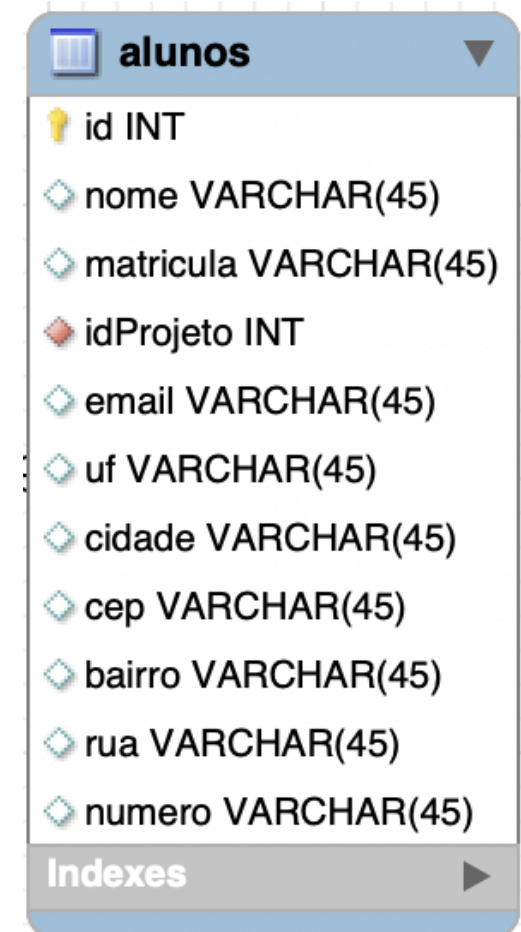
1

@Embeddable

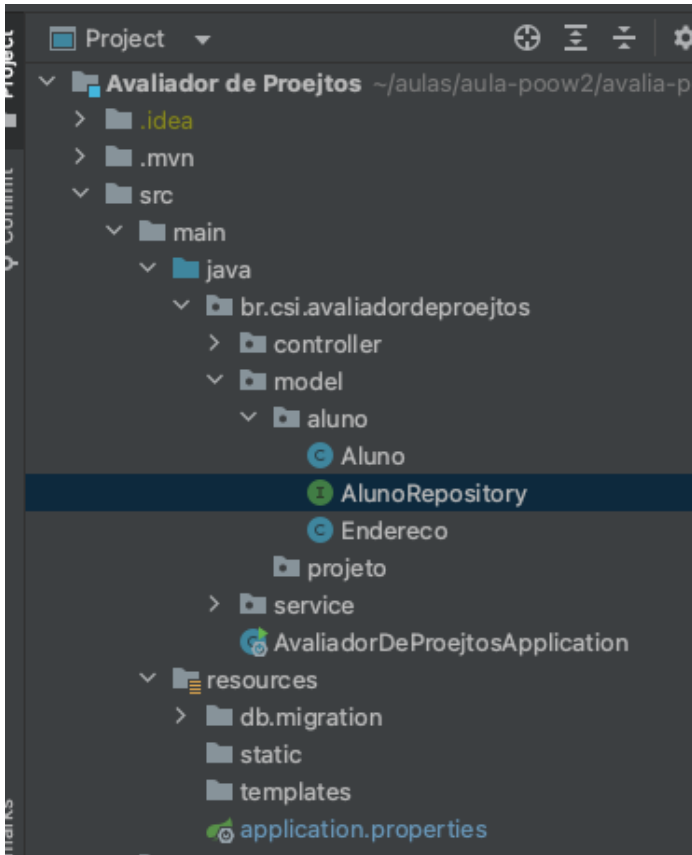


ORM

Paradigma Relacional



Spring Data JPA



```
public class Aluno {  
    private Long id;  
    private UUID uuid;  
    private String nome;  
    private String matricula;  
    private String email;  
    private Endereco endereco;  
  
    //Setters and Getters  
    //Constructors
```

```
@Entity  
@Table(name = "alunos")  
@Getter  
@Setter  
@NoArgsConstructor  
@AllArgsConstructor  
public class Aluno {  
  
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    @UuidGenerator  
    private UUID uuid;  
    private String nome;  
    private String matricula;  
    private String email;  
    @Embedded  
    private Endereco endereco;  
  
}
```

```
public class Endereco {  
  
    private String complemento;  
    private String bairro;  
    private String cep;  
    private String numero;  
    private String cidade;  
    private String uf;  
  
    //Setters and Getters  
    //Constructors
```

```
@Embeddable  
@Getter  
@Setter  
@NoArgsConstructor  
@AllArgsConstructor  
public class Endereco {  
  
    private String complemento;  
    private String bairro;  
    private String cep;  
    private String numero;  
    private String cidade;  
    private String uf;  
  
}
```

Spring Data JPA

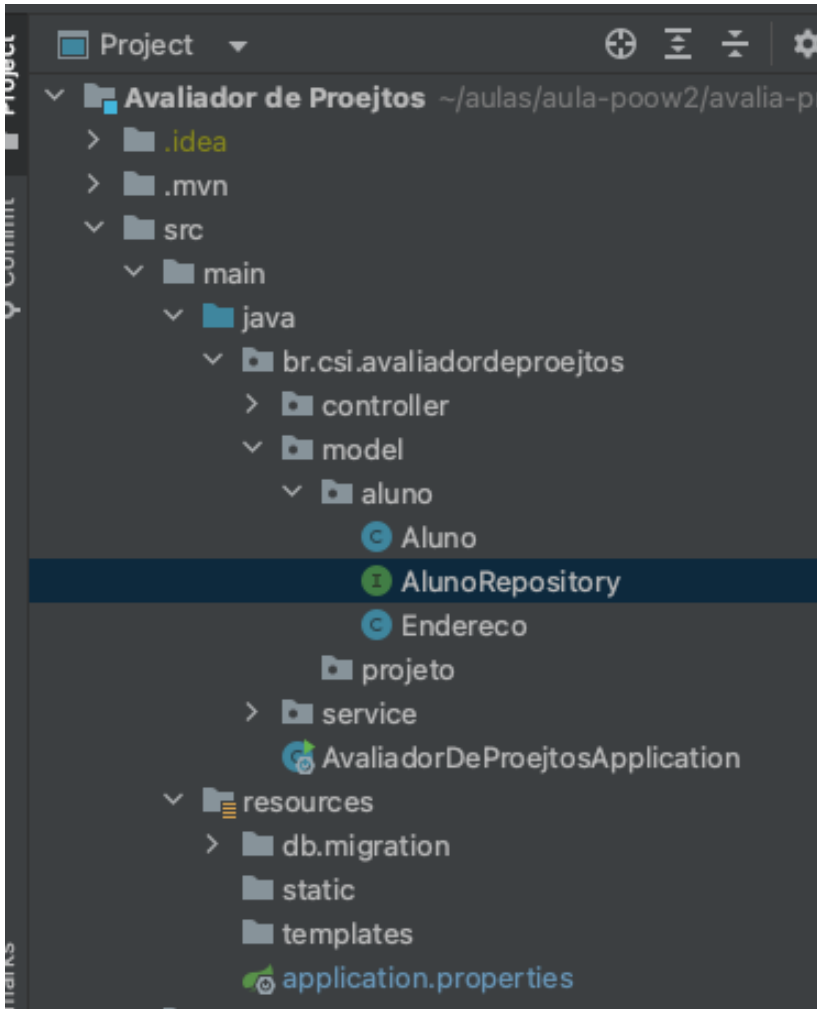
- Repository
 - Interface que contém métodos para manipular Entidades que devem ser persistidas no banco de dados.
 - JpaRepository é a especificação JPA para repositórios. Herda as implementações de CrudRepository e PagingAndSortingRepository
- Metodos
 - findBys ...

<https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

<https://www.baeldung.com/spring-data-repositories>

<https://medium.com/@bubu.tripathy/best-practices-creating-repository-interfaces-with-jpa-d904bee64397>

Spring Data JPA



```
AlunoRepository.java x
1 package br.csi.avaliadordeproejtos.model.aluno;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 3 usages alencar.machado
6 public interface AlunoRepository extends JpaRepository<Aluno, Long> {
7 }
8
```

Persistir dados

Testes

Spring Data JPA: Testar

```
public class Aluno {

    @Id @GeneratedValue(
        strategy =
            GenerationType.IDENTITY)
    private Long id;
    @UuidGenerator
    private UUID uuid;
    private String nome;
    private String matricula;
    private String email;
    @Embedded
    private Endereco endereco;
    public Aluno(String nome, String email,
        String matricula, Endereco endereco){
        this.nome = nome;
        this.email = email;
        this.endereco = endereco;
        this.matricula = matricula;
    }
}
```

```
@Entity
@Table(name = "alunos")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@RequiredArgsConstructor
public class Aluno {

    @Id @GeneratedValue(
        strategy =
            GenerationType.IDENTITY)
    private Long id;
    @UuidGenerator
    private UUID uuid;
    @NotNull
    private String nome;
    @NotNull
    private String matricula;
    @NotNull
    private String email;
    @Embedded
    @NotNull
    private Endereco endereco;
    /* public Aluno(String nome, String email,
        String matricula, Endereco endereco){
        this.nome = nome;
        this.email = email;
        this.endereco = endereco;
        this.matricula = matricula;
    }*/
}
```

Spring Data JPA: Testar Aplicação

```
@SpringBootApplication
public class AvaliadorDeProjetosApplication {
    public static void main(String[] args) {
        SpringApplication.run(AvaliadorDeProjetosApplication.class, args);}

    @Bean
    public CommandLineRunner demo(AlunoRepository repository){
        return (args) ->{
            Endereco en = new Endereco( complemento: "1345", bairro: "Medianeira",
                cep: "97010340", numero: "11", cidade: "Santa Maria", uf: "RS");

            repository.save(new Aluno( nome: "José",
                matricula: "189344", email: "pietra@gmail", en));
        };
    }
}
```

```

@Service
public class AlunoService {

    private final AlunoRepository repository;

    public AlunoService(AlunoRepository repository) { this.repository = repository; }

    public void salvar(Aluno aluno) { this.repository.save(aluno); }
    public List<Aluno> listar() {
        return this.repository.findAll();
    }
    public Aluno getAluno(Long id) {
        return this.repository.findById(id).get();
    }
    public void excluir(Long id) { this.repository.deleteById(id); }

    /*
        Para atualizar uma entidade do banco é necessário pegar a referência desta
        entidade e atualizar com os dados que vieram por parametro.
        O save(...) detecta que esse RECURSO já existe no banco de dados pela busca por id
        assim ao executar o save com id ele faz um UPDATE
    */
    public void atualizar(Aluno aluno){
        Aluno a = this.repository.getReferenceById(aluno.getId());
        a.setNome(aluno.getNome());
        a.setEmail(aluno.getEmail());
        a.setMatricula(aluno.getMatricula());
        a.setEndereco(aluno.getEndereco());
        this.repository.save(a);
    }
}

```